Ex. NO : 06 Date : 13/8/2026

Error detection & correction using Hamming code.

Aim :

To write a program to implement error detection and correction using the hamming code.

Algorithm :

*. Start the program.

*. Read the number of 4 bit data blocks(n) forming the input data stream.

*. For each 4 bit data block D=$d_1 d_2 d_3 d_4$ perform hamming encoding:

a> Arrange 7 bits with a parity bits at position 1, 2, 3 and 4 and data bits at positions 3, 5, 6 & 7.

b> Compute the parity bits using even parity :

$P_1 = d_1$ XOR d_2 XOR d_4 (covers bit pos 1, 3, 5, 7)

$P_2 = d_1$ XOR d_3 XOR d_4 (covers bit pos 2, 3, 6, 7)

$P_4 = d_2$ XOR d_3 XOR d_4 (covers bit pos 4, 5, 6, 7)

c) From the 7-bit hamming code word: $P_1 P_2 d_1 P_4 d_2 d_3 d_4$.

*. Simulate the transmission by introducing a single bit error at a choosen bit position in the code word.

* At the receiver, recompute the check bits (syndrome) from the received code word.

$C_1 = P_1$ XOR d_1 XOR d_2 XOR d_4

$C_2 = P_2$ XOR d_1 XOR d_3 XOR d_4

$C_4 = P_4$ XOR d_2 XOR d_3 XOR d_4

*. compute the error position = $C_4 C_2 C_1$ (binary number formed from the check bits)

*. If error position = 0, no error occured, else the bit at "error position" is inverted to correct the error.

*. Extract the original 4 data bits from the corrected 7-bit code word.

*. Repeat steps 3 to 8 for all n blocks in the data stream.

*. Stop the program.

Code:

```
def encode(d):
    h = [0] * 7
    h[2] = d[0]
    h[4] = d[1]
```

```
    h[5] = d[2]
    h[6] = d[3]
    h[0] = h[2] ^ h[4] ^ h[6]
    h[1] = h[2] ^ h[5] ^ h[6]
    h[3] = h[4] ^ h[5] ^ h[6]
    return h
def detect_and_correct(h):
    c1 = h[0] ^ h[2] ^ h[4] ^ h[6]
    c2 = h[1] ^ h[2] ^ h[5] ^ h[6]
    c4 = h[3] ^ h[4] ^ h[5] ^ h[6]
    error_pos = c1*1 + c2*2 + c4*4
    if error_pos != 0:
        print("Error detected at the position:",
                                    error_pos)
        print("correcting ....")
        h[error_pos-1] ^= 1
    else:
        print("No error detected")
    return error_pos
def to_str(bits):
    return "".join(str(b) for b in bits)
def main():
    print("== HAMMING CODE - ERROR DETECTION
        AND CORRECTION ==\n")
    n = int(input("Enter number of 4-bit
            data blocks in the steam: "))
```

```
for b in range (1, n+1):
    print(f"\n --- Block {b} ---")
    d = list(map(int, input("Enter 4 data
        bits (space separated, e.g. 1 0 1 1): ",
                                    split()))
    h = encode(d)
    print(f"Original Data Bits : {to_str(d)}")
    print(f"Encoded Hamming code (7 bits):
                                {to_str(h)}")
    pos = int(input("Enter bit position to
Introduce error (1-7), 0=no error): "))
    if (1 <= pos <= 7):
        h[pos - 1] ^= 1
        print(f"Transmitted code (with error
Included at pos {pos}): {to_str(h)}")
    else:
        print(f"Transmitted code (no error
            Induced): {to_str(h)}")
    Print(f"Received code : {to_str(h)}")
    print("Checking for errors....")
    detect_and_correct(h)
    Print(f"Corrected code : {to_str(h)}")
    rd = [h[2], h[4], h[5], h[6]]
    Print(f"Recovered Data Bits : {to_str(rd)}")
if __name__ == "__main__":
    main()
```

Output:

```
===HAMMING CODE - ERROR DETECTION AND CORRECTION==
Enter number of 4bit data blocks in the stream: 2
---Block 1---
Enter 4 data bits (space separated, e.g. 1011): 1011
Original Data Bits : 1011
Encoded Hamming code (7 bits): 0110011
Enter bit position to introduce error (1-7) 0 = no
                                        error ): 3
Transmitted code (with error induced at pos 3): 0100011
Received code : 0100011
checking for errors.....
   Error detected at bit position: 3 -> correcting..
corrected code : 0110011
Recovered Data Bits : 1011

---Block 2---
Enter 4 data bits (space separated, e.g 1011): 0110
original Data Bits : 0110
Encoded Hamming code (7 bits) : 1100110
Enter bit position to introduce error (1-7, 0=no
                                        error): 6
Transmitted code (with error induced at pos 6):
                                        1100100
Received code : 1100100
checking for errors....
   Error detected at bit position: 6 -> correcting..
corrected code : 1100110
Recovered Data Bits : 0110
```

Result:

The program to implement error detection and correction using the Hamming code concept was successfully written in python, executed and verified using a sample input data stream.